

SAMUEL MONTESSI DO AMARAL SILVA - 202465563C

SOCKETS
DCC042 - REDES DE COMPUTADORES

JUIZ DE FORA

2026

SAMUEL MONTESSI DO AMARAL SILVA - 202465563C

SOCKETS

DCC042 - REDES DE COMPUTADORES

Atividade realizada como parte da avaliação da disciplina de Redes de Computadores do Departamento de Ciência da Computação da UFJF.
Orientador: Professor Dr. Alex Borges Vieira.

JUIZ DE FORA

2026

“Computadores são como bicicletas para nossas mentes.”

Steve Jobs

SUMÁRIO

1. INTRODUÇÃO	5
2. DESENVOLVIMENTO	6
3. CONCLUSÃO	7
4. REFERÊNCIAS	8

1. INTRODUÇÃO

Processos que rodam em máquinas diferentes comunicam-se enviando mensagens para sockets, sendo o socket a interface de programação que liga a aplicação à camada de transporte. Sobre uma conexão TCP, que funciona como uma tubulação virtual entre o socket do cliente e o socket de conexão do servidor, o transporte garante a entrega ordenada de cada byte e um mesmo servidor consegue manter várias conexões em uma única porta porque cada socket TCP é identificado por uma tupla de quatro elementos, formada pelos endereços IP e portas de origem e destino.

Partindo de um servidor de ECHO apresentado em aula, este trabalho teve dois objetivos: torná-lo capaz de atender vários clientes ao mesmo tempo e definir um protocolo de aplicação simples, com os comandos ECHO, que recebe uma mensagem como parâmetro, e QUIT, que encerra a conexão.

2. DESENVOLVIMENTO

A solução separou as duas responsabilidades: a thread principal apenas aceita conexões, enquanto o atendimento foi encapsulado na classe `ClientHandler`, que implementa `Runnable`. Em vez de criar uma thread por conexão, o que consumiria tempo e memória a cada cliente, usou-se um pool fixo criado com `Executors.newFixedThreadPool`, no qual as threads são criadas uma única vez e reaproveitadas; cada conexão aceita é submetida ao pool com `submit`, e as tarefas excedentes aguardam na fila do executor.

O protocolo definido é textual e orientado a linhas: `ECHO <mensagem>` devolve `OK <mensagem>`, `QUIT` devolve `BYE` e encerra a conexão, e entradas inválidas devolvem mensagens `ERR` sem derrubar a sessão, o que atende aos elementos que um protocolo de camada de aplicação deve especificar, isto é, tipos de mensagens e regras de envio e resposta.

3. CONCLUSÃO

Por fim, os testes evidenciaram, porém, que cada cliente ocupa uma thread durante toda a sessão, ainda que ocioso, o que faz do tamanho do pool o limite de clientes simultâneos. Uma possível melhoria seria definir um tempo limite de inatividade e, para um volume muito maior de conexões, adotar entrada e saída não bloqueantes.

4. REFERÊNCIAS

KUROSE, J.; ROSS, K., Redes de Computadores e a Internet: uma abordagem top-down, 2010, 5ª edição Pearson